

Лабораторная работа №10

Отчёт

Коровкин Никита Михайлович

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Ответ на контрольные вопросы	14
5	Выводы	15
	Список литературы	16

Список иллюстраций

3.1	команда lspci -k	7
3.2	модули ядра	8
3.3	проверка модуля	8
3.4	загрузка модуля	8
3.5	модуль загружен	9
3.6	модуль	9
3.7	модуль не выгружается	10
3.8	модуль загружен	10
3.9	модуль загружен	10
3.10	модуль загружен	11
3.11	выгружаю модуль	11
3.12	список пакетов	11
3.13	обновление - подготовка	12
3.14	обновление	12
3.15	p	12
3.16	проверка версии ядра	13

Список таблиц

1 Цель работы

Получить навыки работы с утилитами управления модулями ядра операционной системы

2 Задание

1. Продемонстрируйте навыки работы по управлению модулями ядра (см. раздел 10.4.1).
2. Продемонстрируйте навыки работы по загрузке модулей ядра с параметрами

3 Выполнение лабораторной работы

Я запустил терминал и получил полномочия администратора, введя команду `su -`. После этого я посмотрел, какие устройства подключены к системе и какие модули ядра с ними связаны, используя команду `lspci -k`. (рис. ??)

```
Definition Audio Controller (rev 01)
  Subsystem: Red Hat, Inc. QEMU Virtual Machine
  Kernel driver in use: snd_hda_intel
  Kernel modules: snd_hda_intel
00:04.0 USB controller: NEC Corporation uPD720200 USB 3.0 Host Controller (rev 0
3)
  Subsystem: Red Hat, Inc. QEMU Virtual Machine
  Kernel driver in use: xhci_hcd
00:05.0 USB controller: Red Hat, Inc. QEMU XHCI Host Controller (rev 01)
  Subsystem: Red Hat, Inc. Device 1100
  Kernel driver in use: xhci_hcd
00:06.0 SCSI storage controller: Red Hat, Inc. Virtio block device
  Subsystem: Red Hat, Inc. Device 0002
  Kernel driver in use: virtio-pci
00:07.0 Communication controller: Red Hat, Inc. Virtio console
  Subsystem: Red Hat, Inc. Device 0003
  Kernel driver in use: virtio-pci
00:08.0 Unclassified device [0002]: Red Hat, Inc. Virtio filesystem
  Subsystem: Red Hat, Inc. Device 0009
  Kernel driver in use: virtio-pci
00:09.0 Unclassified device [00ff]: Red Hat, Inc. Virtio RNG
  Subsystem: Red Hat, Inc. Device 0004
  Kernel driver in use: virtio-pci
```

Рис. 3.1: команда `lspci -k`

В выводе отобразились все устройства — сетевой адаптер, контроллер USB и другие. . Затем я вывел список всех загруженных модулей ядра с помощью команды `lsmod | sort`. Эта команда показала, какие модули сейчас используются системой, а также их взаимозависимости. (рис. ??)

```

snd_hwdep          20480  1 snd_hda_codec
snd_intel_dspcfg    24576  1 snd_hda_intel
snd_pcm            147456  3 snd_hda_intel,snd_hda_codec,snd_hda_core
snd_seq            110592  7 snd_seq_dummy
snd_seq_device      16384  1 snd_seq
snd_seq_dummy       12288  0
snd_timer          49152  3 snd_seq,snd_hrtimer,snd_pcm
soundcore           16384  1 snd
uinput             24576  1
vfat                24576  1
virtio_blk          36864  4
virtio_console      40960  2
virtio_dma_buf      12288  1 virtio_gpu
virtio_gpu           81920  2
virtio_mmio          20480  0
virtio_net          102400  0
vmw_vmci            102400  1 vmw_vsock_vmci_transport
vmw_vsock_virtio_transport_common 53248 1 vsock_loopback
vmw_vsock_vmci_transport 40960 0
vsock               57344  5 vmw_vsock_virtio_transport_common,vsock_loopback,vmw_vsock_vmci_transport
vsock_loopback      12288  0
xfs                 2461696  2

```

Рис. 3.2: модули ядра

Чтобы проверить, загружен ли модуль ext4, я ввёл команду `lsmod | grep ext4`. (рис. ??)

```

# lsmod | grep ext4
#

```

Рис. 3.3: проверка модуля

Для загрузки модуля я использовал команду `modprobe ext4`, после чего снова проверил список модулей. (рис. ??)

```

# lsmod | grep ext4
modprobe ext4

```

Рис. 3.4: загрузка модуля

Теперь ext4 появился в выводе, значит модуль успешно загружен. (рис. ??)


```

root@localhost:~# lsmod | grep ext4
ext4                954368  0
mbcache             20480  1 ext4
jbd2                188416  1 ext4

```

Рис. 3.5: модуль загружен

Далее я посмотрел подробную информацию о модуле ext4 с помощью команды `modinfo ext4`. Перечислю вывод - Rocky kernel signing key - это имя ключа, которым подписан модуль ext4. `sig_key: 24:2F:36:2D:03:F4:44:18:02:8C:20:2B:26:07:EC:2A:2C:64:31:51`

Это отпечаток (fingerprint) открытого ключа, которым модуль был подписан. Он используется для проверки, что подпись сделана доверенным ключом. `sig_hashalgo: sha256` — алгоритм хеширования, применяемый при создании подписи. `signature:` используется ядром Linux для проверки, что модуль не был изменён после компиляции и что он подписан доверенным ключом.(рис. ??)

```

signer:             Rocky kernel signing key
sig_key:            24:2F:36:2D:03:F4:44:18:02:8C:20:2B:26:07:EC:2A:2C:64:31:51
sig_hashalgo:      sha256
signature:          0B:DC:0C:26:5E:C6:CD:85:E1:DC:3B:90:85:24:08:37:96:7C:C2:43:
                    C4:96:F2:64:6F:1F:2B:FA:6B:37:04:F4:76:B9:8E:AD:5B:27:A4:67:
                    5A:9D:43:1D:70:1F:58:A8:0F:FB:31:8F:F8:F6:3D:5D:9B:5B:CE:81:
                    98:2D:C4:82:A1:3D:54:B1:77:73:37:51:D2:FC:FB:79:8D:BB:47:B3:
                    E8:2F:85:F1:B9:49:7E:16:23:85:9B:60:89:8F:69:9A:3B:26:FE:A4:
                    DD:86:1C:84:DD:12:6B:57:4B:B4:28:74:B1:16:2A:01:43:2A:B0:62:
                    7E:70:69:37:2C:20:D6:8E:46:78:B9:06:F4:D3:F4:8A:C8:8F:A9:8E:
                    9E:CD:AE:A7:AC:9B:0E:CC:B6:AA:E2:73:E0:7D:78:BC:6C:17:2A:31:
                    9A:C4:94:54:90:D2:83:C9:06:F0:96:51:29:C4:16:F6:26:D1:2A:98:
                    9A:EE:D9:5D:DB:87:1A:BB:C3:D4:A1:8F:D5:56:79:F1:C8:B6:9C:46:
                    32:35:AF:24:3B:E2:D6:EB:3F:58:76:DA:F0:09:A0:42:00:AE:26:22:
                    55:F7:B4:D7:F5:EC:32:57:4B:1A:CB:81:06:65:5B:B5:EF:5E:32:08:
                    4F:90:50:21:CE:24:32:DC:BF:66:BF:46:F8:F2:42:63:71:F4:7B:28:
                    3B:15:12:60:A9:84:B6:9A:4C:C6:4C:F8:53:A4:8F:B1:D3:AC:95:67:
                    8D:19:29:A2:66:F1:26:AE:7B:1F:37:96:72:E5:A8:01:43:41:8F:EC:
                    77:B5:8D:9E:AA:F0:08:A4:43:76:48:4C:74:96:FD:50:5B:56:DB:68:
                    BA:FA:70:58:59:63:A3:9A:C1:DE:4B:10:C4:6B:1A:9F:95:85:36:B9:
                    58:22:31:3B:7E:A2:40:F8:97:21:BA:F4:E2:36:D2:69:B3:E9:58:93:
                    42:AA:16:30:68:40:4C:8F:4B:5C:96:68:41:A2:AA:44:70:D4:4D:5B:
                    1E:BE:5E:E6

```

Рис. 3.6: модуль

После этого я попробовал выгрузить модуль ext4 командой `modprobe -r ext4`. Иногда ядро не позволяет выгрузить модуль, если он используется системой (например, когда файловая система ext4 используется для текущего корневого

раздела). Поэтому команда может не сработать с первого раза. В этом случае система выдаёт сообщение о том, что модуль занят. Я также попробовал выгрузить модуль xfs командой `modprobe -r xfs`. (рис. ??)

```
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r xfs
modprobe: FATAL: Module xfs is in use.
root@localhost:~#
```

Рис. 3.7: модуль не выгружается

Затем я снова запустил терминал от имени администратора и проверил, загружен ли модуль Bluetooth `lsmod | grep bluetooth`.

Так как он не был загружен, я добавил его командой `modprobe bluetooth`, после чего убедился, что модуль действительно появился в списке активных. (рис. ??)

```
root@localhost:~# lsmod | grep bluetooth
root@localhost:~# modprobe bluetooth
```

Рис. 3.8: модуль загружен

Модуль присутствует. Чтобы узнать, какие ещё модули связаны с Bluetooth, я снова использовал `lsmod | grep bluetooth`. В выводе появилось `rfkill` (рис. ??)

```
root@localhost:~# lsmod | grep bluetooth
bluetooth          909312  0
rfkill              36864  4 bluetooth
```

Рис. 3.9: модуль загружен

Затем я посмотрел информацию о самом модуле bluetooth с помощью `modinfo bluetooth`. Здесь также отображаются ключ, которыми подписан этот модуль и присутствуют его параметры. (рис. ??)

```
signature:      11:56:48:F7:C0:BF:16:5F:84:E9:4D:9E:0B:6F:A6:12:81:5A:6E:57:
                26:44:2C:7B:6F:88:42:2D:9D:9D:BC:D0:E8:95:B0:4D:B6:0E:3F:38:
                38:97:EF:8F:6D:9E:7F:00:F6:B9:E3:3C:CA:E0:39:44:2D:CD:5C:07:
                E0:5A:91:88:5D:78:C9:C3:73:AD:EB:E7:51:CE:AE:F1:FB:F9:B0:98:
                2C:10:B6:99:8E:33:64:4F:22:FA:4C:8B:0B:E8:B7:7C:5D:46:E6:32:
                FC:0A:4A:D0:61:09:7C:1E:59:E8:C2:3F:FC:92:1E:DC:A2:67:C5:CD:
                38:E5:66:FA:6A:59:DF:C9:0E:7F:B9:15:AD:90:6D:33:B1:FC:E9:7C:
                09:7D:CF:38:84:50:D0:AB:BB:8F:82:FC:01:3D:CF:AC:42:65:1D:07:
                27:84:EE:F0:E9:A0:55:72:3C:0C:04:47:83:31:D4:5A:61:60:AB:55:
                6B:7B:58:DB:68:C4:40:0E:81:2A:C3:5A:3E:05:84:51:0F:63:89:09:
                0B:21:8C:33:34:C8:23:99:4D:41:D7:D1:E1:F2:F9:10:7F:0C:97:18:
                82:4D:55:67:5F:35:68:DF:E4:F4:20:77:8D:8D:5C:38:01:6D:6D:35:
                6A:96:5A:C0:C0:1B:95:AA:81:72:59:40:2B:2D:98:DE:05:D8:32:C4:
                D4:1E:61:00:FF:64:22:67:7F:C2:F9:14:77:F8:42:79:EE:96:BC:1C:
                67:70:C6:11:8E:61:39:22:76:ED:D3:76:3B:8B:75:91:03:9E:55:15:
                77:00:52:C5:7F:13:16:5B:19:90:5A:A9:6E:30:9E:4C:C4:57:E9:EB:
                14:34:82:FB:D1:9D:68:99:83:CF:F5:DF:A7:D4:FC:65:58:3F:BC:F9:
                96:3A:98:23:FA:54:0E:F4:44:75:4E:5E:71:D5:CD:15:A2:6E:50:CF:
                39:18:47:50:14:AB:F5:26:23:05:09:BF:B6:CF:50:C5:05:31:9A:97:
                8E:41:D6:0D
parm:          disable_esco:Disable eSCO connection creation (bool)
parm:          disable_ertm:Disable enhanced retransmission mode (bool)
parm:          enable_ecred:Enable enhanced credit flow control mode (bool)
```

Рис. 3.10: модуль загружен

После проверки я выгрузил модуль командой `modprobe -r bluetooth`. (рис. ??)

```
parm:          enable_ecred:Enable enhanced
root@localhost:~# modprobe -r bluetooth
root@localhost:~#
```

Рис. 3.11: выгружаю модуль

Далее я снова открыл терминал с правами администратора и посмотрел текущую версию ядра с помощью команды `uname -r`. После этого я вывел список всех пакетов, относящихся к ядру, используя `dnf list kernel`. (рис. ??)

```
root@localhost:~# uname -r
6.12.0-55.40.1.el10_0.aarch64
root@localhost:~# dnf list kernel
Rocky Linux 10 - Bas [ === ] --- B/s | 0 B --- ETA
```

Рис. 3.12: список пакетов

Чтобы убедиться, что система полностью обновлена, я выполнил `dnf upgrade --refresh`. Это важно перед обновлением ядра, чтобы избежать конфликтов. (рис. ??)

```
root@localhost:~# dnf upgrade --refresh
```

Рис. 3.13: обновление - подготовка

После этого я обновил ядро и саму систему.(рис. ??)

```
root@localhost:~# dnf update kernel
dnf update
dnf update
dnf upgrade --refresh
Последняя проверка окончания срока действия метаданных: 0:00:59 назад, Ср 05 ноя
2025 09:40:28.
```

Рис. 3.14: обновление

Когда обновление завершилось, я перезагрузил компьютер и при загрузке выбрал новое ядро в меню.(рис. ??)

```
*Rocky Linux (6.12.0-55.41.1.el10_0.aarch64) 10.0 (Red Quartz)
Rocky Linux (6.12.0-55.40.1.el10_0.aarch64) 10.0 (Red Quartz)
Rocky Linux (6.12.0-55.12.1.el10_0.aarch64) 10.0 (Red Quartz)
Rocky Linux (0-rescue-4961ea2ca17f4df1ade7b7579e3cec25) 10.0 (Red Quartz)
UEFI Firmware Settings
```

Рис. 3.15: п

После входа в систему я снова проверил версию ядра через `uname -r` — она изменилась, значит обновление прошло успешно. Команда `hostnamectl` также показывает текущую версию ядра и другую системную информацию.(рис. ??)

```
nmkorovkin@localhost:~$ uname -r
hostnamectl
6.12.0-55.41.1.el10_0.aarch64
  Static hostname: (unset)
  Transient hostname: localhost
    Icon name: computer-vm
    Chassis: vm 🖥️
    Machine ID: 4961ea2ca17f4df1ade7b7579e3cec25
    Boot ID: a5403502589e420686ee7bc1d907da9e
  Virtualization: qemu
  Operating System: Rocky Linux 10.0 (Red Quartz)
    CPE OS Name: cpe:/o:rocky:rocky:10::baseos
    OS Support End: Thu 2035-05-31
OS Support Remaining: 9y 6month 3w 2d
    Kernel: Linux 6.12.0-55.41.1.el10_0.aarch64
    Architecture: arm64
    Hardware Vendor: QEMU
    Hardware Model: QEMU Virtual Machine
    Firmware Version: 0.0.0
    Firmware Date: Fri 2015-02-06
    Firmware Age: 10y 8month 4w 1d
nmkorovkin@localhost:~$
```

Рис. 3.16: проверка версии ядра

4 Ответ на контрольные вопросы

1. Какая команда показывает текущую версию ядра? ☒ Команда `uname -r`.
2. Как можно посмотреть более подробную информацию о текущей версии ядра? ☒ С помощью команды `hostnamectl`, которая показывает не только версию ядра, но и данные о системе, хосте и архитектуре.
3. Какая команда показывает список загруженных модулей ядра? ☒ Команда `lsmod`.
4. Какая команда позволяет определять параметры модуля ядра? ☒ Команда `modinfo имя_модуля`.
5. Как выгрузить модуль ядра? ☒ Командой `modprobe -r имя_модуля`.
6. Что можно сделать, если при попытке выгрузить модуль появляется ошибка? ☒ Ошибка обычно означает, что модуль используется системой. В этом случае нужно сначала остановить процесс или отключить устройство, которое использует модуль, а затем повторить команду.
7. Как определить, какие параметры модуля ядра поддерживаются? ☒ Командой `modinfo имя_модуля` — в выводе перечислены все доступные параметры и их описание.
8. Как установить новую версию ядра? ☒ Сначала обновить пакеты системы (`dnf upgrade --refresh`), затем выполнить `dnf update kernel`, перезагрузить систему и выбрать новое ядро при загрузке.

5 Выводы

В ходе выполнения данной лабораторной работы были получены навыки управления ядром.

Список литературы